

工业视觉 workflow 平台开发者手册

v14.4 多用户工作区权限版 - 面向维护、扩展和故障定位

项目包	industrial_vision_layout_routing_v14_multiuser_workspace.zip
生成对象	当前源码：industrial_vision_refactored_error_trace_v2
主要读者	图像算法开发者、平台维护者、后端/前端协作者
重点目标	扩展公共图像节点；理解运行链路；快速定位并修复平台问题
验证状态	基于当前源码执行 pytest：76 passed, 2 warnings（Python 3.13 fork 弃用警告）

本手册按“先总览、再扩展、最后排障”的顺序组织。前半部分帮助你理解平台架构和数据流；中间部分详细讲解如何新增公共图像节点；后半部分提供权限、模型、项目、任务、前端 UI 和运行错误的定位方法。

目录

1. 平台定位与核心概念	5
1.1 总体技术栈	5
1.2 运行链路总图	5
2. 目录结构与维护地图	6
2.1 关键代码规模	7
2.2 文件系统数据目录	7
3. 开发环境、启动与基础检查	7
3.1 安装依赖	7
3.2 启动平台	7
3.3 最小健康检查	7
4. 后端架构详解	8
4.1 FastAPI 应用创建	8
4.2 API 路由清单	8
4.3 用户、会话和工作区权限	9
4.4 依赖函数与权限校验入口	10
4.5 核心服务层逐文件说明	10
5. workflow 数据结构与执行器	11
5.1 WorkflowRequest JSON 协议	11
5.2 节点、连线、模块模型	11
5.3 单图执行算法	11
5.4 LogicIfElse 动态分支	11
5.5 节点结果规范	12
5.6 批量流程执行	12
5.7 节点错误响应结构	12
6. 算子系统与统一注册表	12
6.1 算子函数签名	13

6.2 OperatorDefinition 和 OperatorRegistry	13
6.3 节点元数据字段规范	13
6.4 当前算子清单	14
6.5 全量算子元数据详情	16
7. 公共图像节点模块化结构	21
7.1 公共辅助函数	22
7.2 新增公共图像节点的完整步骤	22
7.3 示例：新增 GammaCorrection 图像节点	22
7.4 新节点测试模板	23
7.5 扩展图像节点时最常见错误	23
8. ORB、YOLO 与高级图像能力	24
8.1 ORBFeatureMatch 维护重点	24
8.2 YOLO 模型管理与缓存	24
9. Python 代码片段节点	25
9.1 运行变量与输出约定	25
9.2 安全限制	25
10. 项目、版本、JSON、模型、数据集和任务	25
10.1 项目保存 vs JSON 文件	25
10.2 ProjectStore 版本机制	26
10.3 模型资产与节点调用	26
10.4 数据集和回归测试	26
11. 前端架构与 UI 维护	26
11.1 前端 API 调用约定	27
11.2 模块化工作流 UI	27
11.3 工作流结构树显示原则	27
11.4 前端状态对象维护要点	28
12. API 与权限维护手册	28

12.1 认证 API	28
12.2 工作区 API	28
12.3 项目、模型、运行和任务 API	28
13. 独立脚本导出机制	29
14. 自定义算子插件机制	29
15. 测试体系与回归命令	30
16. 快速故障定位指南	30
16.1 后端运行失败定位顺序	30
16.2 前端 UI 问题定位	31
16.3 节点执行错误分析模板	31
16.4 常见异常与原因	31
17. 生产化建议与技术债清单	31
17.1 维护者日常检查清单	32
18. 附录 A：后端函数/类索引	32
19. 附录 B：关键 JSON 样例	34
19.1 登录与工作区请求	34
19.2 项目保存请求	34
19.3 模型元数据修改	34
20. 附录 C：快速扩展流程图	35
21. 附录 D：当前版本确认	35

1. 平台定位与核心概念

- 这是一个基于 FastAPI 后端和原生 JavaScript 前端的工业视觉 workflow 平台。它通过节点和连线描述图像处理、AI 推理、逻辑判断、批量处理和自定义 Python 代码。
- 平台当前已经支持多用户登录、工作区权限、项目保存/版本、模型资产、数据集、回归测试、运行历史、异步任务以及 workflow 模块化 UI。
- 模块化 UI 不改变后端执行语义；它只是前端画布和 workflow JSON 中的组织单元，用于折叠、分区、显示结构。真正执行仍以 nodes + edges 为准。

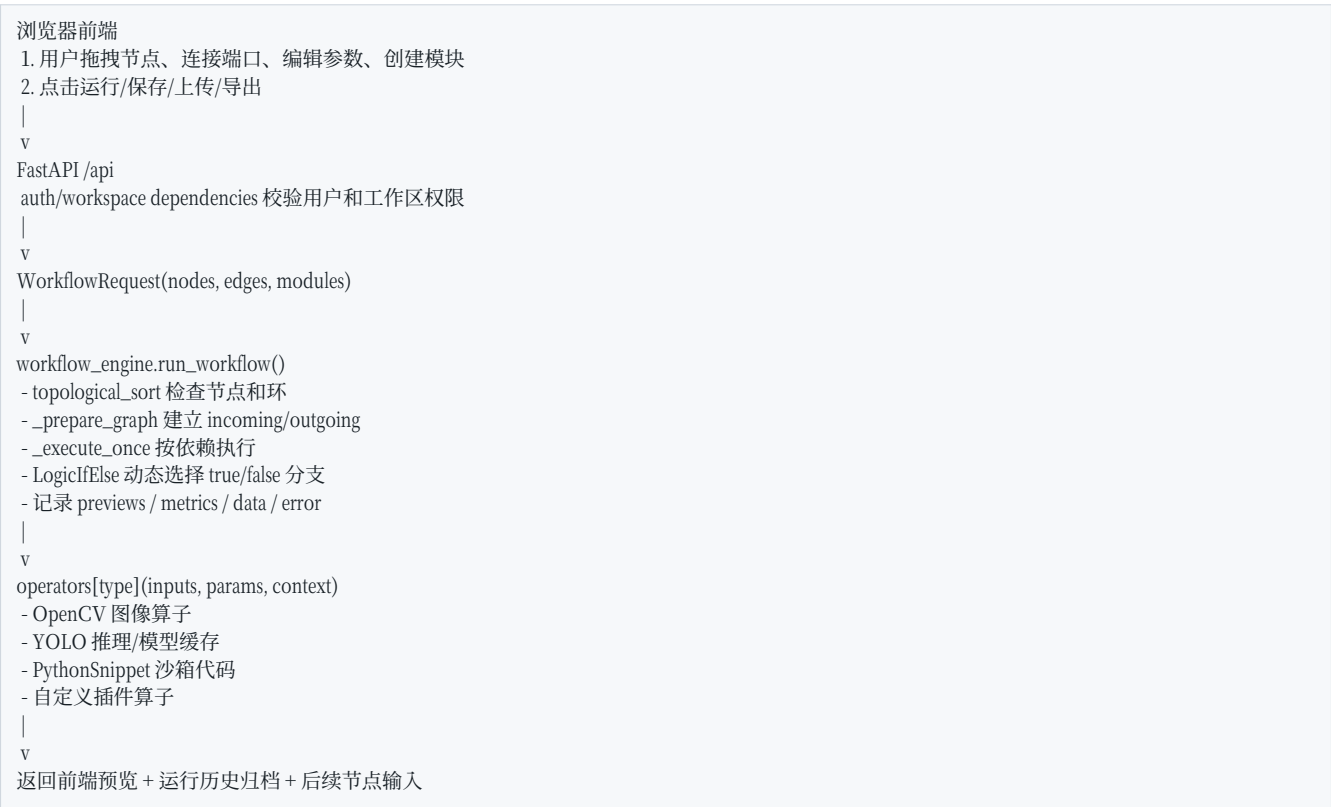
开发者维护时最重要的四个“源头”是：算子运行函数、节点元数据、统一注册表、执行器。新增公共图像节点时，必须让这四个层次一致。

概念	含义	主要文件
节点 Node	画布上的一个处理单元，例如 Threshold、YOLO、PythonSnippet。	backend/models/workflow.py； frontend/assets/js/app/canvas.js
连线 Edge	节点之间的端口连接，定义数据流和执行依赖。	backend/models/workflow.py；workflow_engine.py
模块 Module	前端可折叠的功能区域，保存标题、颜色、包含节点，不改变执行。	WorkflowModule；canvas.js
项目 Project	服务端保存的工程，含 workflow、版本、发布状态。	backend/services/project_store.py
模型 Model	YOLO 等 AI 节点使用的模型资产和缓存对象。	backend/algorithms/yolo/model_manager.py
工作区 Workspace	用户隔离边界，同一工作区共享项目/模型/数据集。	backend/services/team_store.py
算子 Operator	后端实际执行函数，签名为 operator(inputs, params, context) -> dict。	backend/algorithms/*

1.1 总体技术栈

层	技术	职责
前端	HTML + CSS + 原生 JavaScript	节点拖拽、画布渲染、参数编辑、模块折叠、项目/模型/工作区交互。
后端 API	FastAPI + Pydantic	路由、权限、上传、执行、导出、项目/模型/数据集/任务服务。
图像算法	OpenCV + NumPy	公共图像节点、ORB、滤波、阈值、边缘、形态学、轮廓。
AI 推理	Ultralytics YOLO（按需安装）	检测、分割、分类、姿态、OBB 推理和后处理。
持久化	当前为 JSON/文件系统	项目、版本、工作区、用户、数据集、运行历史、模型元数据。
测试	pytest	后端执行器、节点、YOLO、项目模型、多用户隔离、前端布局检查。

1.2 运行链路总图



2. 目录结构与维护地图

维护平台时，不建议从入口文件盲目搜索。下面的维护地图直接说明每类问题应该看哪些文件。

路径	作用	维护关注点
run.py	本地启动入口；调用 backend.main:create_app()	端口、reload、uvicorn 启动问题从这里看。
backend/main.py	FastAPI 应用创建、状态对象初始化、静态前端挂载	所有服务 store 的初始化位置；确认路由和静态资源路径。
backend/config.py	项目根目录、上传目录、模型目录、上传限制、缓存参数	新增目录或修改上传限制时先改这里。
backend/models/workflow.py	WorkflowRequest、Node、Edge、Module 的 Pydantic 数据结构	前后端 JSON 协议改变必须同步这里。
backend/services/workflow_engine.py	workflow 执行器、拓扑排序、If/Else 动态分支、批量流程、节点错误封装	平台运行错误定位首要文件。
backend/algorithms/image/*.py	公共 OpenCV 图像算子分模块实现	扩展公共图像节点主要修改这里。
backend/algorithms/metadata.py	节点元数据、参数 schema、前端节点库展示配置	新增/改节点后前端是否能展示，主要看这里。
backend/algorithms/registry.py	运行函数、元数据、目录、插件统一注册	新增内置算子需要导入并加入 _BUILTIN_OPERATORS。
backend/services/script_exporter.py	工作流导出独立 Python 脚本	新增算子若需要导出，需补编译逻辑。
backend/api/routes/*.py	FastAPI 路由：项目、模型、数据集、任务、认证、工作区等	API 报错或权限问题从这里看 Depends。
backend/services/*_store.py	文件系统版持久化服务	项目/模型/数据集/运行历史/登录/工作区数据落盘逻辑。
frontend/assets/js/app/canvas.js	画布、节点、模块、连线、属性面板核心交互	前端 UI 问题和模块折叠/拖动主要看这里。
frontend/assets/js/app/actions.js	运行、保存、打开、导入导出、模型/项目动作	按钮点击后没反应优先看这里。
frontend/assets/js/app/auth_workspace.js	登录、工作区切换、在线用户显示	多人在线和权限头注入相关。
frontend/assets/js/app/python_snippet_ide.js	Python 代码片段 IDE 面板	代码片段交互体验和校验结果展示。

路径	作用	维护关注点
tests/*.py	回归测试	修改核心逻辑后必须跑 pytest。

2.1 关键代码规模

当前项目已经进入中型前后端应用规模。下面列出主要文件规模，便于判断重构风险。

区域	约代码行数	说明
backend	8677 行	后端 API、算法、服务、执行器、认证和存储。
frontend/assets/js/app	3795 行	前端状态、画布、动作、模块 UI、Python IDE、登录工作区。
tests	2099 行	回归测试覆盖执行器、节点、YOLO、多用户隔离等。
docs	1316 行	已有功能文档和版本说明。

2.2 文件系统数据目录

目录	内容	排障用途
uploads/	上传的普通图片。	ReadImage / upload 失败时检查文件是否存在和扩展名。
models/uploads/	上传的模型文件和 models.json 元数据。	YOLO 节点找不到模型、模型权限错误、缓存异常时检查。
projects/	按工作区隔离的项目 JSON 和版本目录。	项目列表、保存、回滚、发布问题。
workspaces/	工作区成员、角色、Owner 信息。	403 权限、工作区不可见、成员不生效。
auth/	users.json 和 sessions.json。	登录失败、token 过期、用户信息不正确。
datasets/	数据集和样本元数据。	回归测试样本找不到或期望值不匹配。
run_history/	运行历史和结果摘要。	某次运行结果追溯、错误复盘。
jobs/	当前版本主要由进程内 JobQueue 管理，目录预留。	异步任务持久化增强时使用。

3. 开发环境、启动与基础检查

3.1 安装依赖

```
cd industrial_vision_refactored_error_trace_v2
python -m venv .venv
source .venv/bin/activate # Windows 使用 .venv\Scripts\activate
pip install -r requirements.txt

# 如果要运行 YOLO 节点，需要确认 ultralytics 已安装；测试里部分 YOLO 逻辑会使用 mock。
pip install ultralytics
```

3.2 启动平台

```
python run.py
# 或者
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000

# 浏览器访问
http://127.0.0.1:8000/
```

3.3 最小健康检查

检查项	命令/操作	预期结果
前端页面	打开 /	加载 index.html，不报 404。
算子元数据	GET /api/operators	返回 OPERATOR_CATALOG、OPERATOR_META 和插件加载错误列表。
登录注册	POST /api/auth/register 后 POST /api/auth/login	获得 token，前端顶部显示用户和工作区。
工作流运行	添加 ReadImage + Display 后运行	返回 success=true 和 previews。
测试套件	pytest -q	当前源码验证为 76 passed，2 warnings。

4. 后端架构详解

4.1 FastAPI 应用创建

后端入口是 `backend.main:create_app()`。它创建 FastAPI app，把各种 store 和 manager 挂到 `application.state` 上，然后以 `/api` 前缀挂载后端路由，以 `/assets` 挂载前端静态资源。

```
application.state.auth_store = AuthStore(AUTH_DIR)
application.state.image_store = ImageStore(UPLOAD_DIR, allowed_extensions=ALLOWED_IMAGE_EXTENSIONS)
application.state.model_manager = DEFAULT_MODEL_MANAGER
application.state.project_store = ProjectStore(PROJECTS_DIR)
application.state.workspace_store = WorkspaceStore(WORKSPACES_DIR)
application.state.dataset_store = DatasetStore(DATASETS_DIR)
application.state.run_history_store = RunHistoryStore(RUN_HISTORY_DIR)
application.state.job_queue = JobQueue(max_workers=2)
application.include_router(api_router, prefix="/api")
```

如果某个 API 依赖的服务为空或行为异常，优先确认 `create_app()` 是否创建了对应 state，以及依赖函数是否从 `request.app.state` 读取到了正确对象。

4.2 API 路由清单

模块	HTTP	完整路径	函数	权限/说明
auth.py	POST	/api/auth/register	register	公开/内部校验
auth.py	POST	/api/auth/login	login	公开/内部校验
auth.py	POST	/api/auth/logout	logout	公开/内部校验
auth.py	GET	/api/auth/session	session	登录/兼容头
datasets.py	GET	/api/datasets	list_datasets	view
datasets.py	POST	/api/datasets	create_dataset	edit
datasets.py	GET	/api/datasets/{dataset_id}	get_dataset	view
datasets.py	PUT	/api/datasets/{dataset_id}	update_dataset	edit
datasets.py	POST	/api/datasets/{dataset_id}/samples	add_samples	edit
datasets.py	DELETE	/api/datasets/{dataset_id}	delete_dataset	manage
export_script.py	POST	/api/export-script	export_script	公开/内部校验
jobs.py	GET	/api/jobs	list_jobs	view
jobs.py	GET	/api/jobs/{job_id}	get_job	view
jobs.py	POST	/api/jobs/run	submit_workflow_job	run
models.py	GET	/api/models	list_models	view
models.py	GET	/api/models/assets	list_model_assets	view
models.py	PATCH	/api/models/{model_id}	update_model_metadata	model

模块	HTTP	完整路径	函数	权限/说明
models.py	DELETE	/api/models/{model_id}	delete_model	model
models.py	POST	/api/models/cache/clear	clear_model_cache	model
operators.py	GET	/api/operators	list_operators	公开/内部校验
projects.py	GET	/api/projects	list_projects	view
projects.py	POST	/api/projects	save_project	edit
projects.py	GET	/api/projects/{project_id}	get_project	view
projects.py	DELETE	/api/projects/{project_id}	delete_project	manage
projects.py	GET	/api/projects/{project_id}/versions	list_project_versions	view
projects.py	GET	/api/projects/{project_id}/versions/{version_id}	get_project_version	view
projects.py	GET	/api/projects/{project_id}/diff	diff_project_versions	view
projects.py	POST	/api/projects/{project_id}/rollback/{version_id}	rollback_project	edit
projects.py	POST	/api/projects/{project_id}/release/{version_id}	release_project_version	manage
python_snippet.py	POST	/api/python-snippet/validate	validate_python_snippet	公开/内部校验
regression.py	POST	/api/regression-tests/run	run_regression_test	run
regression.py	POST	/api/regression-tests/jobs	submit_regression_job	run
runs.py	GET	/api/runs	list_runs	view
runs.py	GET	/api/runs/{run_id}	get_run	view
team.py	GET	/api/users/me	get_me	登录/兼容头
team.py	GET	/api/workspaces	list_workspaces	公开/内部校验
team.py	POST	/api/workspaces	create_workspace	公开/内部校验
team.py	GET	/api/workspaces/{workspace_id}	get_workspace	公开/内部校验
team.py	POST	/api/workspaces/{workspace_id}/members	upsert_member	公开/内部校验
team.py	DELETE	/api/workspaces/{workspace_id}/members/{member_user_id}	remove_member	公开/内部校验
team.py	POST	/api/presence/heartbeat	heartbeat	登录/兼容头
team.py	GET	/api/workspaces/{workspace_id}/presence	list_presence	公开/内部校验
upload.py	POST	/api/upload	upload_image	公开/内部校验
upload.py	POST	/api/upload-model	upload_model	model
workflow.py	POST	/api/run	execute_workflow	run

4.3 用户、会话和工作区权限

当前版本新增了真实登录会话。请求优先使用 Authorization: Bearer 获取用户；为了兼容旧版本，本地工具仍可使用 X-User-Id / X-User-Email。工作区通过 X-Workspace-Id 或 query 参数 workspace_id 指定，未指定时使用用户默认个人工作区。

角色	权限集合	典型用途
Owner	view, edit, run, manage, delete, release, model, member	工作区所有者，最高权限。
Maintainer	view, edit, run, manage, release, model, member	算法负责人，管理发布和成员。
Developer	view, edit, run, model	日常开发者，可编辑项目和上传模型。

角色	权限集合	典型用途
Reviewer	view, run	评审者，可查看和运行。
Viewer	view	只读查看。
Operator	view, run	现场操作人员，只运行已配置流程。
# 请求头示例 Authorization: Bearer <login_token> X-Workspace-Id: ws_xxx # 兼容旧本地模式（未登录时） X-User-Id: alice X-User-Email: alice@example.com X-Workspace-Id: ws_alice		

4.4 依赖函数与权限校验入口

依赖函数	含义	常见报错
get_current_user	解析 Bearer token 或兼容 X-User-Id。	401 登录状态失效。
get_active_workspace_id	从 X-Workspace-Id/query/default_workspace_id 获取当前工作区。	工作区 ID 未传时回到个人工作区。
require_workspace_view	需要 view 权限。	项目/模型/运行历史列表不可见。
require_workspace_edit	需要 edit 权限。	保存项目、修改数据集失败。
require_workspace_run	需要 run 权限。	运行工作流、提交任务、回归测试失败。
require_workspace_model	需要 model 权限。	上传/删除/修改模型失败。
require_workspace_manage	需要 manage 权限。	删除项目、发布/管理操作失败。

4.5 核心服务层逐文件说明

services 目录是后端业务状态的核心。当前多数服务使用文件系统 JSON 持久化，便于直接部署和调试；生产环境可逐步替换为数据库/Redis/对象存储，但对外方法可以尽量保持不变。

服务	文件	核心方法	维护重点
AuthStore	backend/services/auth_store.py	register, authenticate, get_user_by_token, logout	PBKDF2 密码哈希、token 过期、users.json/sessions.json。
WorkspaceStore	backend/services/team_store.py	ensure_personal_workspace, list_workspaces, require_permission, touch_presence	角色权限、工作区成员、在线用户 TTL。
ImageStore	backend/services/image_store.py	save, resolve_path, read, list_images, prepare_output_directory	上传图片、路径白名单、批量输入/输出目录。
ProjectStore	backend/services/project_store.py	save_project, list_versions, diff_versions, rollback_project, release_version	项目版本、回滚、发布、工作区隔离目录。
DatasetStore	backend/services/dataset_store.py	create_dataset, list_datasets, add_samples, update_dataset	样本元数据、期望结果、回归测试输入。
RunHistoryStore	backend/services/run_history_store.py	record_run, list_runs, get_run	运行记录摘要、失败复盘、历史结果归档。
JobQueue	backend/services/job_queue.py	submit, list_jobs, get	进程内异步任务，生产建议替换为 Redis 队列。
YoloModelManager	backend/algorithms/yolo/model_manager.py	load_model, register_upload, list_models, update_model_metadata, delete_model	模型资产、工作区权限、LRU/TTL 缓存。

5. workflow 数据结构与执行器

5.1 WorkflowRequest JSON 协议

前端运行、保存、导入导出的核心都是 WorkflowRequest。项目保存会在此基础上增加 project_id、version、workspace_id 等外层字段，但执行器只关心 nodes、edges、modules 和 image_id。

```
{
  "version": "2",
  "image_id": null,
  "nodes": [
    {"id": "n1", "type": "ReadImage", "position": {"x": 100, "y": 100}, "params": {"image_path": "samples/a.png"}},
    {"id": "n2", "type": "Threshold", "position": {"x": 380, "y": 100}, "params": {"threshold": 127, "max_value": 255}}
  ],
  "edges": [
    {"id": "e1", "source": "n1", "target": "n2", "sourceHandle": "out", "targetHandle": "in"}
  ],
  "modules": [
    {"id": "m1", "name": "定位模块", "node_ids": ["n1", "n2"], "color": "#3B82F6", "collapsed": false}
  ]
}
```

5.2 节点、连线、模块模型

模型	字段	说明
WorkflowNode	id, type, position, params, collapsed	type 必须存在于 OPERATORS；params 与 OPERATOR_META[type].params 合并使用。
WorkflowEdge	id, source, target, sourceHandle, targetHandle	source/target 必须是现有节点；handle 用于多端口节点。
WorkflowModule	id, name, node_ids, description, color, collapsed	只用于 UI 组织和版本对比，不参与执行排序。
ExecutionContext	image_id, image_store, current_image_path, batch_index, user_id, workspace_id, model_manager	算子运行时上下文；YOLO 和批量处理依赖它。

5.3 单图执行算法

```
run_workflow(workflow)
if workflow 包含 BatchReadImages 或 SaveBatchResults:
    return run_batch_workflow(...)
context = ExecutionContext(...)
order, outputs, previews = _execute_once(...)
sink_data = 收集末端节点 data
return {success, mode="single", previews, order, data}

_execute_once(workflow)
1. topological_sort 检查重复节点、无效连线、环
2. 建立 nodes / incoming / outgoing
3. 从 sink 节点反向触发上游执行
4. 每个节点收集输入端口 inputs[targetHandle]
5. 调用 OPERATORS[node.type](inputs, node.params, context)
6. 规范化 out_color_space、metrics、data、previews
7. 任何节点异常包装为 WorkflowExecutionError
```

5.4 LogicIfElse 动态分支

If/Else 是执行器中特殊处理的节点。它不会静态执行 true_value 和 false_value 两个分支，而是先运行 condition，上游值转为 bool 后，仅执行选中的分支。这一点对性能和错误隔离很重要。

LogicIfElse 输入端口：
 condition 必须且只能连接一个
 true_value 条件为真时执行
 false_value 条件为假时执行

执行逻辑：
 execute_node(condition.source)
 selected_handle = "true_value" if condition else "false_value"
 execute_node(selected_handle.source)
 run_current_node(LogicIfElse, [condition_edge, selected_handle])

5.5 节点结果规范

返回键	类型	用途
out	np.ndarray 或任意值	主输出端口；图像节点通常放图像。
out_color_space	BGR/RGB/GRAY	图像颜色空间；未提供时执行器自动推断。
metrics	dict	节点自有指标，显示在预览中，并写入 data 的节点指标段。
data	dict	结构化数据，会继承上游并传给下游，批量 CSV 会展开。
其他端口名	任意 JSON 安全值或图像	用于多输出端口，如 detections、mask、roi、confidence。

metrics 和 data 不能直接放 np.ndarray。若把图像数组放入 metrics/data，执行器会在节点阶段抛出可定位错误，避免 FastAPI 序列化阶段才变成不可定位的 500。

5.6 批量流程执行

批量流程有严格结构：必须且只能包含一个 BatchReadImages，一个 SaveBatchResults，不能同时包含 ReadImage。保存节点必须是末端节点且只有一个上游图像输入。

```
BatchReadImages(folder_path, extensions, recursive, max_images, continue_on_error)
-> 对文件夹内图片逐张构造 ExecutionContext(current_image_path=source_path)
-> 对同一 workflow 执行 _execute_once(collect_previews=False)
-> SaveBatchResults 接收最终图像并保存到 output_dir
-> result.data 被 flatten 后写入 CSV
-> 最后一张成功图再执行一次 collect_previews=True 用于前端预览
```

5.7 节点错误响应结构

WorkflowExecutionError 会保留失败前已经成功的节点预览，前端可以据此高亮失败节点和未执行节点。这是排障最重要的数据结构之一。

```
{
  "success": false,
  "error": "节点 n3（固定阈值 / Threshold）执行失败：缺少输入连线",
  "mode": "single",
  "stage": "node_execution",
  "order": ["n1", "n2", "n3", "n4"],
  "executed_order": ["n1", "n2"],
  "pending_nodes": ["n4"],
  "failed_node": {
    "node_id": "n3",
    "type": "Threshold",
    "label": "固定阈值",
    "error": "...",
    "cause_type": "ValueError"
  },
  "previews": ["成功节点预览...", "失败节点错误预览"]
}
```

6. 算子系统与统一注册表

6.1 算子函数签名

```
from backend.algorithms.types import OperatorInputs, OperatorParams, ExecutionContext, OperatorResult

def op_xxx(inputs: OperatorInputs, params: OperatorParams, context: ExecutionContext) -> OperatorResult:
    image = inputs.get("in")
    # 读取 params, 执行算法
    return {
        "out": output_image,
        "metrics": {"some_score": 0.95},
        "data": {"debug": "..."},
    }
```

参数	来源	说明
inputs	上游节点输出按 targetHandle 收集	例如 inputs["in"]、inputs["template"]、inputs["mask"]。
params	节点自身参数	来自前端属性面板，默认值在 metadata.py。
context	执行上下文	包含用户、工作区、批量当前图片、模型管理器、ImageStore。

6.2 OperatorDefinition 和 OperatorRegistry

v14 系列将运行函数、元数据、目录分组和脚本导出编译器统一到 OperatorDefinition。旧变量 OPERATORS、OPERATOR_META、OPERATOR_CATALOG 仍保留兼容。

```
@dataclass(frozen=True)
class OperatorDefinition:
    type_name: str
    operator: Operator
    metadata: dict[str, Any]
    category: str
    group: str
    script_compiler: ScriptCompiler | None = None
    builtin: bool = True

OPERATOR_REGISTRY.register_many(_BUILTIN_OPERATORS, _BASE_OPERATOR_META, _BASE_OPERATOR_CATALOG)
OPERATORS = OPERATOR_REGISTRY.operators
OPERATOR_META = OPERATOR_REGISTRY.metadata
OPERATOR_CATALOG = OPERATOR_REGISTRY.catalog
```

6.3 节点元数据字段规范

字段	必填性	说明
label	建议必填	前端显示名称。
category	建议必填	一级分类，如 图像、逻辑。
group	建议必填	节点库分组，如 滤波降噪、模板匹配。
kind	可选	节点语义，如 io、color、filter、segment、match、ai、script。
has_input / has_output	旧式节点常用	没有 inputs/outputs 明细时，前端按默认 in/out 处理。
inputs	多端口节点必填	列表，元素含 handle、label、type、required。
outputs	多输出节点必填	列表，元素含 handle、label、type。
params	建议必填	默认参数。
param_schema	建议复杂参数必填	前端控件类型、选项、提示、上传 API。
description	建议必填	节点帮助说明。
hidden	可选	隐藏旧兼容节点，如 Gray/BGR2RGB。
result_type	可选	预览类型提示，metrics/image/auto。

6.4 当前算子清单

分类	分组	Type	标签	输入	输出	关键参数	实现文件
图像	输入输出	ReadImage	读取图像	-	out	image_path	backend/algorithms/image/io.py
图像	输入输出	Display	结果显示	in	out	-	backend/algorithms/image/io.py
图像	批量处理	BatchReadImages	批量读取图片	-	out	folder_path, recursive, extensions, max_images, continue_on_error	backend/algorithms/image/io.py
图像	批量处理	SaveBatchResults	保存批量结果	in	-	output_dir, save_images, image_format, filename_suffix, preserve_structure, write_csv, csv_filename	backend/algorithms/image/io.py
图像	颜色与增强	Gray	灰度化	in	out	-	backend/algorithms/image/color.py
图像	颜色与增强	BGR2RGB	BGR转RGB	in	out	-	backend/algorithms/image/color.py
图像	颜色与增强	ColorConvert	颜色转换	in	out	mode	backend/algorithms/image/color.py
图像	颜色与增强	Invert	反相	in	out	-	backend/algorithms/image/color.py
图像	颜色与增强	Normalize	归一化	in	out	-	backend/algorithms/image/color.py
图像	颜色与增强	EqualizeHist	直方图均衡	in	out	-	backend/algorithms/image/color.py
图像	颜色与增强	CLAHE	CLAHE增强	in	out	clip_limit, tile_grid	backend/algorithms/image/color.py
图像	滤波降噪	MeanBlur	均值滤波	in	out	ksize	backend/algorithms/image/filters.py
图像	滤波降噪	GaussianBlur	高斯滤波	in	out	ksize, sigma	backend/algorithms/image/filters.py
图像	滤波降噪	MedianBlur	中值滤波	in	out	ksize	backend/algorithms/image/filters.py
图像	滤波降噪	BilateralFilter	双边滤波	in	out	d, sigma_color, sigma_space	backend/algorithms/image/filters.py
图像	滤波降噪	Sharpen	锐化	in	out	amount	backend/algorithms/image/filters.py
图像	阈值与边缘	Threshold	固定阈值	in	out	threshold, max_value	backend/algorithms/image/thresholding.py
图像	阈值与边缘	OtsuThreshold	Otsu阈值	in	out	-	backend/algorithms/image/thresholding.py

分类	分组	Type	标签	输入	输出	关键参数	实现文件
图像	阈值与边缘	AdaptiveThreshold	自适应阈值	in	out	block_size, c	backend/algorithms/image/thresholding.py
图像	阈值与边缘	Canny	Canny边缘	in	out	threshold1, threshold2	backend/algorithms/image/edges.py
图像	阈值与边缘	Sobel	Sobel梯度	in	out	dx, dy, ksize	backend/algorithms/image/edges.py
图像	阈值与边缘	Laplacian	Laplacian边缘	in	out	ksize	backend/algorithms/image/edges.py
图像	形态学	Erode	腐蚀	in	out	ksize, iterations	backend/algorithms/image/morphology.py
图像	形态学	Dilate	膨胀	in	out	ksize, iterations	backend/algorithms/image/morphology.py
图像	形态学	MorphOpen	开运算	in	out	ksize	backend/algorithms/image/morphology.py
图像	形态学	MorphClose	闭运算	in	out	ksize	backend/algorithms/image/morphology.py
图像	形态学	MorphGradient	形态学梯度	in	out	ksize	backend/algorithms/image/morphology.py
图像	几何变换	Resize	缩放	in	out	scale_percent	backend/algorithms/image/geometry.py
图像	几何变换	Flip	翻转	in	out	flip_code	backend/algorithms/image/geometry.py
图像	几何变换	Rotate90	旋转90度	in	out	direction	backend/algorithms/image/geometry.py
图像	测量分析	FindContours	轮廓查找	in	out	threshold, min_area	backend/algorithms/image/contours.py
图像	模板匹配	ORBFeatureMatch	ORB角点ROI/Mask匹配	in, template, mask	out, mask, roi, roi_mask, masked, annotated, matches, match_count, good_match_count, inlier_count, confidence, homography_found, transform_found, corners, roi_bbox, roi_polygon, homography, affine	template_path, roi_mode, roi_x, roi_y, roi_w, roi_h, mask_path	backend/algorithms/image/template_matching.py
图像	AI推理	YoloInfer	YOLO推理	in	out, detections, count, top_class, top_confidence, task	model_path, task, confidence, iou, imgsz, max_det, max_results	backend/algorithms/yolo/ops.py

分类	分组	Type	标签	输入	输出	关键参数	实现文件
图像	AI推理	YoloSelectTopDetections	YOLO目标筛选	detections, image	selected, count, expected_count, missing_indices, first, second, centers, mask	selection_mode, class_indices, min_confidence, top_k, sort_by, require_exact_count	backend/algorithms/yolo/selection.py
图像	AI推理	RoiPairEdgeDistance	ROI边缘距离	image, detections	out, edge1, edge2, distance, center1, center2	threshold1, threshold2, aperture_size, padding	backend/algorithms/yolo/roi_distance.py
图像	自定义算子	MeanBrightness	平均亮度统计	in	out, mean_brightness, brightness_std	-	backend/custom_operators/mean_brightness.py
逻辑	数据与变量	LogicConstant	常量	-	out	value_type, value	backend/algorithms/logic_ops.py
逻辑	条件判断	LogicCompare	比较	a, b	out	operator	backend/algorithms/logic_ops.py
逻辑	条件判断	LogicBoolean	布尔运算	a, b	out	operation	backend/algorithms/logic_ops.py
逻辑	条件判断	LogicNot	NOT 取反	in	out	-	backend/algorithms/logic_ops.py
逻辑	流程选择	LogicIfElse	If / Else 选择	condition, true_value, false_value	out	-	backend/algorithms/logic_ops.py
逻辑	循环控制	LogicForLoop	For 循环	in	out	initial_value, iterations, update_operation, step	backend/algorithms/logic_ops.py
逻辑	循环控制	LogicWhileLoop	While 循环	in	out	initial_value, compare_operator, compare_value, update_operation, step, max_iterations	backend/algorithms/logic_ops.py
逻辑	脚本扩展	PythonSnippet	Python代码片段	in, aux, value	out, image, value, data	code, timeout_ms, isolated, output_kind, output_color_space	backend/algorithms/python_snippet_ops.py

6.5 全量算子元数据详情

下表展开每个算子的输入端口、输出端口和默认参数。维护时若前端显示、连线、执行结果不一致，优先对照这里和 metadata.py。

Type	标签/说明	输入端口	输出端口	参数默认值
ReadImage	读取图像 从服务器可访问的文件路径读取图片。 分类: 图像/输入输出	-	out:image	{ "image_path": "" }

Type	标签/说明	输入端口	输出端口	参数默认值
Display	结果显示 显示并传递上游结果。 分类: 图像/输入输出	in:image	out:image	-
BatchReadImages	批量读取图片 枚举文件夹内图片，并依次把每张图片送入后续算法节点。 分类: 图像/批量处理	-	out:image	{ "folder_path": "", "recursive": "YES", "extensions": ".jpg,.jpeg,.png,.bmp,.tif,.tiff,.webp", "max_images": 0, "continue_on_error": "YES" }
SaveBatchResults	保存批量结果 保存每张处理结果图，并把源路径、文件名及所有数据结果汇总到 CSV。 分类: 图像/批量处理	in:image	-	{ "output_dir": "batch_outputs", "save_images": "YES", "image_format": "SOURCE", "filename_suffix": "_result", "preserve_structure": "YES", "write_csv": "YES", "csv_filename": "results.csv" }
Gray	灰度化 旧版兼容节点，加载时会迁移为 ColorConvert(mode=GRAY)。 分类: 图像/颜色与增强	in:image	out:image	-
BGR2RGB	BGR转RGB 旧版兼容节点，加载时会迁移为 ColorConvert(mode=RGB)。 分类: 图像/颜色与增强	in:image	out:image	-
ColorConvert	颜色转换 通过参数选择灰度化或 BGR 转 RGB。 分类: 图像/颜色与增强	in:image	out:image	{ "mode": "GRAY" }
Invert	反相 像素取反。 分类: 图像/颜色与增强	in:image	out:image	-
Normalize	归一化 拉伸到 0-255。 分类: 图像/颜色与增强	in:image	out:image	-
EqualizeHist	直方图均衡 灰度直方图均衡化。 分类: 图像/颜色与增强	in:image	out:image	-
CLAHE	CLAHE增强 局部对比度增强。 分类: 图像/颜色与增强	in:image	out:image	{ "clip_limit": 2.0, "tile_grid": 8 }

Type	标签/说明	输入端口	输出端口	参数默认值
MeanBlur	均值滤波 均值平滑。 分类: 图像/滤波降噪	in:image	out:image	{ "ksize": 5 }
GaussianBlur	高斯滤波 高斯平滑, 常用于边缘检测前降噪。 分类: 图像/滤波降噪	in:image	out:image	{ "ksize": 5, "sigma": 0 }
MedianBlur	中值滤波 适合去椒盐噪声。 分类: 图像/滤波降噪	in:image	out:image	{ "ksize": 5 }
BilateralFilter	双边滤波 保边去噪。 分类: 图像/滤波降噪	in:image	out:image	{ "d": 7, "sigma_color": 75, "sigma_space": 75 }
Sharpen	锐化 增强边缘与细节。 分类: 图像/滤波降噪	in:image	out:image	{ "amount": 1.0 }
Threshold	固定阈值 灰度二值化。 分类: 图像/阈值与边缘	in:image	out:image	{ "threshold": 127, "max_value": 255 }
OtsuThreshold	Otsu阈值 自动阈值分割。 分类: 图像/阈值与边缘	in:image	out:image	-
AdaptiveThreshold	自适应阈值 适合光照不均场景。 分类: 图像/阈值与边缘	in:image	out:image	{ "block_size": 11, "c": 2 }
Canny	Canny边缘 经典边缘检测。 分类: 图像/阈值与边缘	in:image	out:image	{ "threshold1": 50, "threshold2": 150 }
Sobel	Sobel梯度 一阶梯度边缘。 分类: 图像/阈值与边缘	in:image	out:image	{ "dx": 1, "dy": 0, "ksize": 3 }
Laplacian	Laplacian边缘 二阶边缘检测。 分类: 图像/阈值与边缘	in:image	out:image	{ "ksize": 3 }
Erode	腐蚀 收缩白色区域。 分类: 图像/形态学	in:image	out:image	{ "ksize": 3, "iterations": 1 }
Dilate	膨胀 扩张白色区域。 分类: 图像/形态学	in:image	out:image	{ "ksize": 3, "iterations": 1 }

Type	标签/说明	输入端口	输出端口	参数默认值
MorphOpen	开运算 先腐蚀后膨胀，常用于去小噪点。 分类: 图像/形态学	in:image	out:image	{ "ksize": 3 }
MorphClose	闭运算 先膨胀后腐蚀，常用于补小孔。 分类: 图像/形态学	in:image	out:image	{ "ksize": 3 }
MorphGradient	形态学梯度 提取形态学边缘。 分类: 图像/形态学	in:image	out:image	{ "ksize": 3 }
Resize	缩放 按百分比缩放图像。 分类: 图像/几何变换	in:image	out:image	{ "scale_percent": 50 }
Flip	翻转 1水平, 0垂直, -1双向。 分类: 图像/几何变换	in:image	out:image	{ "flip_code": 1 }
Rotate90	旋转90度 0顺时针, 1逆时针, 2旋转180。 分类: 图像/几何变换	in:image	out:image	{ "direction": 0 }
FindContours	轮廓查找 二值化后查找轮廓并绘制包围框。 分类: 图像/测量分析	in:image	out:image	{ "threshold": 127, "min_area": 20 }
ORBFeatureMatch	ORB角点ROI/Mask匹配 使用 ORB 角点匹配定位模板, 并把模板图上的矩形 ROI 或不规则 mask 通过单应/仿射矩阵映射到待检测图, 直接输出 ROI/mask/抠图结果。 分类: 图像/模板匹配	in:image template:image 可选 mask:image 可选	out:image mask:image roi:image roi_mask:image masked:image annotated:image matches:image match_count:number good_match_count:number inlier_count:number confidence:number homography_found:boolean transform_found:boolean corners:list roi_bbox:list roi_polygon:list homography:list affine:list	{ "template_path": "", "roi_mode": "FULL", "roi_x": 0, "roi_y": 0, "roi_w": 0, "roi_h": 0, "mask_path": "", "mask_threshold": 1, "transform_model": "HOMOGRAPHY", "output_mode": "MASK", "crop_apply_mask": "YES", "n_features": 1000, "scale_factor": 1.2, "n_levels": 8, "fast_threshold": 20, "ratio_test": 0.75, "min_matches": 8, "ransac_reproj_threshold": 5.0, "max_draw_matches": 40 }

Type	标签/说明	输入端口	输出端口	参数默认值
YoloInfer	YOLO推理 选择 YOLO/Ultralytics 模型文件，对图像执行检测、分割、分类、姿态估计或旋转框推理；可按类别 index（如 0,1,8）过滤输出，并把过滤后的 detections 交给后续节点处理。 分类: 图像/AI推理	in:image	out:image detections:list count:number top_class:text top_confidence:number task:text	{ "model_path": " yolov8n.pt", "task": "detect", "confidence": 0.35, "iou": 0.45, "imgsz": 640, "max_det": 300, "max_results": 300, "class_indices": "", "class_filter": "", "include_geometry": "YES", "max_geometry_points": 500, "output_mode": "ANNOTATED", "device": "" }
YoloSelectTopDetections	YOLO目标筛选 从 YOLO detections 中筛选目标。可按类别 index 分别选出每类最高置信度目标，并根据 class_indices 或 top_k 自动生成 mask1、mask2... 等单目标二值 Mask，供后续边缘、形态学或测量节点继续处理。 分类: 图像/AI推理	detections:list image:image 可选	selected:list count:number expected_count:number missing_indices:list first:object second:object centers:list mask:image	{ "selection_mode": "AUTO", "class_indices": "", "min_confidence": 0.35, "top_k": 2, "sort_by": "CONFIDENCE_DESC", "require_exact_count": "YES" }
RoiPairEdgeDistance	ROI边缘距离 接收原图和两个检测框，分别裁剪 ROI 做 Canny 边缘检测，并计算两个 ROI 中心点之间的像素距离。 分类: 图像/AI推理	image:image detections:list	out:image edge1:image edge2:image distance:number center1:list center2:list	{ "threshold1": 50, "threshold2": 150, "aperture_size": 3, "padding": 0 }
MeanBrightness	平均亮度统计 计算图像平均亮度和标准差；批量运行时自动写入 CSV。 分类: 图像/自定义算子	in:image	out:image mean_brightness:number brightness_std:number	-
LogicConstant	常量 产生数值、布尔值或文本常量，供其他逻辑节点使用。 分类: 逻辑/数据与变量	-	out:any	{ "value_type": "NUMBER", "value": "0" }
LogicCompare	比较 比较两个输入值并输出布尔结果。 分类: 逻辑/条件判断	a:number b:number	out:boolean	{ "operator": "GT" }
LogicBoolean	布尔运算 对两个布尔条件执行与、或、异或等运算。 分类: 逻辑/条件判断	a:boolean b:boolean	out:boolean	{ "operation": "AND" }

Type	标签/说明	输入端口	输出端口	参数默认值
LogicNot	NOT 取反 对输入条件执行逻辑取反。 分类: 逻辑/条件判断	in:boolean	out:boolean	-
LogicIfElse	If / Else 选择 根据条件从 If 和 Else 两个输入中 选择一个结果；运行时只执行被 选中的值分支。 分类: 逻辑/流程选择	condition:boolean true_value:any false_value:any	out:any	-
LogicForLoop	For 循环 按固定次数更新一个数值，最多 执行 10000 次。 分类: 逻辑/循环控制	in:number 可选	out:number	{ "initial_value": 0, "iterations": 5, "update_operation": " ADD", "step": 1 }
LogicWhileLoop	While 循环 条件成立时持续更新数值，并受 最大循环次数保护。 分类: 逻辑/循环控制	in:number 可选	out:number	{ "initial_value": 0, "compare_operator": " LT", "compare_value": 10, "update_operation": " ADD", "step": 1, "max_iterations": 100 }
PythonSnippet	Python代码片段 在流程中插入受限 Python 代码。 支持专用开发界面，能查看端口 映射、运行变量、输出约定、模 板和最近运行结果。 分类: 逻辑/脚本扩展	in:any 可选 aux:any 可选 value:any 可选	out:any image:image value:any data:object	{ "code": "# 输入可通过 inputs['in'] / image / aux / value 访问。\\n# 直接给 out / image / value / metrics / data 赋 值即可。\\nout = image.copy() if image is not None else value\\nmetrics = {'ok': True}\\n", "timeout_ms": 1000, "isolated": "YES", "output_kind": " AUTO", "output_color_space": " AUTO" }

7. 公共图像节点模块化结构

你的主要工作是扩展公共图像节点，因此本章是日常维护最重要的部分。image_ops.py 已经不再承载实际算子，它只是兼容入口；真正代码位于 backend/algorithms/image/ 包。

文件	包含算子	建议新增内容
backend/algorithms/image/io.py	ReadImage, BatchReadImages, SaveBatchResults, Display	输入输出、结果保存、读写相关节点
backend/algorithms/image/color.py	ColorConvert, Invert, Normalize, EqualizeHist, CLAHE	颜色空间、对比度、灰度/通道增强

文件	包含算子	建议新增内容
backend/algorithms/image/filters.py	MeanBlur, GaussianBlur, MedianBlur, BilateralFilter, Sharpen	平滑、去噪、锐化和频域滤波
backend/algorithms/image/thresholding.py	Threshold, OtsuThreshold, AdaptiveThreshold	二值化、局部阈值、分割前处理
backend/algorithms/image/edges.py	Canny, Sobel, Laplacian	边缘、梯度、纹理/轮廓前置算子
backend/algorithms/image/morphology.py	Erode, Dilate, MorphOpen, MorphClose, MorphGradient	形态学、骨架/孔洞处理
backend/algorithms/image/geometry.py	Resize, Flip, Rotate90	旋转、平移、仿射、透视、裁剪、坐标变换
backend/algorithms/image/contours.py	FindContours	轮廓、测量、几何特征统计
backend/algorithms/image/template_matching.py	ORBFeatureMatch	模板匹配、ROI/Mask 映射、定位类算法
backend/algorithms/image/common.py	gray_input, bgr_input	新增公共图像类型转换工具函数

7.1 公共辅助函数

函数	文件	用途
gray_input(image)	image/common.py	把 BGR/RGB/GRAY 输入规范为灰度图；阈值、边缘、轮廓类节点常用。
bgr_input(image)	image/common.py	把输入规范为 BGR；绘制标注、保存、OpenCV 彩色操作常用。
ensure_odd(value)	algorithms/helpers.py	滤波核大小必须为奇数时使用。
kernel_from_params(params)	algorithms/helpers.py	形态学核构造。

7.2 新增公共图像节点的完整步骤

- 建议任何新图像节点都按以下 8 步执行。这样前端可展示、后端可运行、导出可选支持、测试可覆盖。
- 确定算法类别，选择 backend/algorithms/image/ 下已有文件；如果是新领域，再新建文件并在 image/__init__.py 导出。
 - 实现 op_xxx(inputs, params, context) 函数；输入缺失时主动 raise ValueError，错误会被执行器定位到当前节点。
 - 返回字典，至少包含 out；需要统计值时返回 metrics；需要结构化输出时返回 data 或独立端口。
 - 在 backend/algorithms/metadata.py 添加 OPERATOR_META[type]，包含 label、category、group、params、param_schema。
 - 在 OPERATOR_CATALOG 的对应 category/group 中加入 type。
 - 在 backend/algorithms/registry.py 导入函数并加入 _BUILTIN_OPERATORS。
 - 若独立脚本导出需要支持，在 backend/services/script_exporter.py 添加编译逻辑或自定义 script_compiler。
 - 新增 tests/test_xxx.py，至少测试正常输出、参数边界、错误输入和前端元数据可见。

7.3 示例：新增 GammaCorrection 图像节点

下面是完整思路示例。实际提交时需要根据项目风格调整。

```
# backend/algorithms/image/color.py
import cv2
import numpy as np
from backend.algorithms.image.common import bgr_input

def op_gamma_correction(inputs, params, context):
    image = inputs.get("in")
    if image is None:
        raise ValueError("缺少输入图像")
    gamma = float(params.get("gamma", 1.0) or 1.0)
    if gamma <= 0:
        raise ValueError("gamma 必须大于 0")
    img = bgr_input(image)
    inv_gamma = 1.0 / gamma
    table = np.array([(i / 255.0) ** inv_gamma * 255 for i in range(256)], dtype=np.uint8)
    out = cv2.LUT(img, table)
    return {"out": out, "out_color_space": "BGR", "metrics": {"gamma": gamma}}

# backend/algorithms/metadata.py
OPERATOR_META["GammaCorrection"] = {
    "label": "Gamma 校正",
    "category": "图像",
    "group": "颜色与增强",
    "kind": "color",
    "has_input": True,
    "has_output": True,
    "params": {"gamma": 1.0},
    "param_schema": {
        "gamma": {"label": "Gamma", "type": "number", "help": "大于 0 ; 小于 1 变暗, 大于 1 变亮。"}
    },
    "description": "基于查找表的 Gamma 亮度校正。",
}

# OPERATOR_CATALOG["图像"] 中颜色与增强 group 的 items 加入 "GammaCorrection"

# backend/algorithms/registry.py
from backend.algorithms.image import op_gamma_correction

_BUILTIN_OPERATORS = {
    # ...
    "GammaCorrection": op_gamma_correction,
}

# backend/algorithms/image/__init__.py
from backend.algorithms.image.color import op_gamma_correction
```

7.4 新节点测试模板

```
import numpy as np
from backend.algorithms.image.color import op_gamma_correction
from backend.algorithms.registry import OPERATOR_META, OPERATORS

class DummyContext:
    pass

def test_gamma_correction_outputs_image():
    img = np.full((20, 20, 3), 128, dtype=np.uint8)
    result = op_gamma_correction({"in": img}, {"gamma": 1.2}, DummyContext())
    assert "out" in result
    assert result["out"].shape == img.shape
    assert result["metrics"]["gamma"] == 1.2

def test_gamma_is_registered():
    assert "GammaCorrection" in OPERATORS
    assert OPERATOR_META["GammaCorrection"]["group"] == "颜色与增强"
```

7.5 扩展图像节点时最常见错误

现象	原因	解决
前端节点库看不到新节点	只写了 op 函数，未改 metadata/catalog/registry。	检查 OPERATOR_META、OPERATOR_CATALOG、_BUILTIN_OPERATORS。
运行时报“未知算子”	前端 JSON 里的 type 没有注册到 OPERATORS。	确认 registry.py 导入和 _BUILTIN_OPERATORS。
节点报“缺少输入连线”	metadata.has_input 或 inputs.required 与实际端口不一致。	多输入节点必须写 inputs 列表，handle 和前端连线一致。
预览颜色异常	BGR/RGB/GRAY 没有正确声明。	返回 out_color_space；使用 bgr_input/gray_input。
FastAPI 500 但无节点定位	metrics/data 中返回了 np.ndarray 或不可 JSON 序列化对象。	图像放 out/独立图像端口；metrics/data 只放 JSON 安全值。
批量 CSV 列异常	data 嵌套结构过复杂或字段名变化。	保持 metrics/data 字段稳定；列表会被 JSON 字符串化。

8. ORB、YOLO 与高级图像能力

8.1 ORBFeatureMatch 维护重点

ORBFeatureMatch 位于 backend/algorithms/image/template_matching.py。它的目标不只是输出匹配可视化，而是把模板图上的 FULL/RECT/MASK 区域通过匹配得到的变换映射到待检测图，并直接输出 mask、roi、masked 等结果。

输入/参数	说明
in	待检测图。
template	模板图，可选；未连接时使用 template_path。
mask	模板侧不规则 mask，可选；未连接时使用 mask_path。
roi_mode	FULL、RECT、MASK。
transform_model	HOMOGRAPHY、AFFINE、AFFINE_PARTIAL。
output_mode	MASK、MASKED、ROI、ROI_MASK、ANNOTATED。
维护此节点时优先检查：模板读取、mask 二值化、ORB 参数、ratio test、RANSAC、warpPerspective/warpAffine、ROI 外接框裁剪、输出端口是否与 metadata 一致。	

8.2 YOLO 模型管理与缓存

YOLO 相关代码已经拆分到 backend/algorithms/yolo/。模型加载由 YoloModelManager 统一负责，支持工作区隔离、LRU 缓存、TTL 过期、上传模型元数据。

文件	职责
yolo/model_manager.py	模型资产元数据、路径解析、工作区校验、缓存、上传记录、删除、清理。
yolo/ops.py	YoloInfer 主算子，调用模型并输出标注图、detections、count 等。
yolo/result_parser.py	解析 detect/segment/classify/pose/obb 的输出。
yolo/selection.py	按类别、置信度、面积、中心等筛选 Top 检测。
yolo/roi_distance.py	两个 ROI/Mask/目标边缘距离测量。
yolo/common.py	类别过滤、输入图转换、task 标准化。
# YOLO 节点推荐调用方式 1. 模型页面或 YOLO 参数面板上传模型，形成 models/uploads/<file> 2. YoloInfer.params.model_path = "models/uploads/xxx.pt" 或直接使用 "yolov8n.pt" 3. 运行时 ExecutionContext 带 user_id、workspace_id、model_manager 4. load_yolo_model / YoloModelManager.load_model 校验模型属于当前工作区 5. 缓存 key 通常由 user/workspace/model/task/device 组成，避免跨工作区复用错误模型	

9. Python 代码片段节点

PythonSnippet 是强扩展能力，也是高风险能力。它支持在工作流中插入自定义 Python 片段，并通过前端 IDE 面板展示输入映射、可用变量、输出约定、模板和最近运行结果。

9.1 运行变量与输出约定

变量/输出	说明
inputs	所有输入端口字典：inputs["in"], inputs["aux"], inputs["value"]。
image	inputs["in"] 快捷别名。
aux	inputs["aux"] 快捷别名。
value	inputs["value"] 快捷别名。
params	节点参数。
context	运行上下文信息的安全子集。
np, cv2, math, statistics, json	内置可用模块。
out, image, value, metrics, data	直接赋值输出；或在 process() 返回字典。

```

# 直接赋值写法
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
out = cv2.Canny(gray, 50, 150)
metrics = {"edge_pixels": int(np.count_nonzero(out))}

def process(inputs, params, context):
    img = inputs.get("in")
    return {"out": cv2.bitwise_not(img), "metrics": {"ok": True}}
```

9.2 安全限制

限制	目的
默认子进程隔离	超时可终止，减少主服务被死循环拖垮。
timeout_ms	限制执行时间，避免长时间阻塞。
AST 安全检查	禁止 import、open、eval、exec、compile、__import__ 等高风险操作。
禁止双下划线名称/属性	降低访问内部对象和逃逸风险。
JSON 安全输出	metrics/data 不能包含图像数组。

如果要部署到真正多人生产环境，建议将 PythonSnippet 迁移到容器/沙箱 worker，限制文件系统、CPU、内存和网络。当前实现适合内网团队试用，不适合暴露给不可信用户。

10. 项目、版本、JSON、模型、数据集和任务

10.1 项目保存 vs JSON 文件

功能	保存位置	用途	是否含版本/权限/运行关联
保存项目	服务端 projects/，按工作区隔离	正式工程保存、版本历史、回滚、发布、协作。	是
保存当前工作流 JSON	浏览器下载的本地文件	离线快照、迁移、发给别人、备份。	通常否
打开项目	从服务端项目库加载	继续开发已有项目，保留 project_id 和版本上下文。	是
打开 JSON 文件	从本地文件导入	临时恢复或迁移，导入后不一定属于原项目。	通常否

10.2 ProjectStore 版本机制

ProjectStore 当前使用文件系统保存项目。每次 save_project 都会生成一个版本快照；支持 list_versions、get_version、diff_versions、rollback_project、release_version。

```
projects/  
  <workspace_id>/  
    <project_id>.json  
    <project_id>_versions/  
      v0001.json  
      v0002.json  
      v0003.json
```

10.3 模型资产与节点调用

模型页面不是单纯展示。它保存模型文件和元数据，YoloInfer 节点通过 model_path 引用模型，运行时由 YoloModelManager 校验当前工作区是否有权使用。

模型字段	作用
model_id	模型资产 ID。
path / relative_path	实际模型文件位置。
workspace_id	隔离边界；不同工作区不能互用。
owner_id	上传者。
task	detect/segment/classify/pose/obb/auto。
class_names	类别表，用于检查工作流和模型兼容性。
status	开发中、已验证、已发布、废弃等。
metrics	mAP、召回率、误检率等模型评价。

10.4 数据集和回归测试

数据集用于将算法开发从“单图调参”升级为“批量回归”。DatasetStore 管理数据集和样本，regression 路由加载项目版本和数据集逐张执行，输出通过率、失败样本、耗时等报告。

组件	文件	说明
DatasetStore	backend/services/dataset_store.py	创建、列表、读取、更新、添加样本、删除数据集。
Regression API	backend/api/routes/regression.py	同步/异步运行回归测试。
RunHistoryStore	backend/services/run_history_store.py	记录每次运行摘要、结果、错误和工作流快照。
JobQueue	backend/services/job_queue.py	进程内异步任务队列；适合小团队，生产建议迁移 Redis/Celery。

11. 前端架构与 UI 维护

前端是无构建工具的原生 JavaScript。优点是部署简单，缺点是全局状态和函数较多。维护 UI 时需要明确每个文件的职责。

文件	职责	常见修改场景
state.js	全局 workflow 状态、节点、连线、模块、tabs、localStorage 兼容。	状态字段新增、导入导出、工作流名称。
state_store.js	状态深拷贝、服务端项目保存/打开封装。	项目保存失败、JSON 格式迁移。
operators.js	根据 /api/operators 渲染节点库。	节点元数据未显示、拖拽节点创建。
canvas.js	画布主逻辑：节点、连线、模块、折叠、拖动、属性面板。	大部分 UI bug。

文件	职责	常见修改场景
actions.js	运行、上传、打开保存项目、导出脚本、模型动作。	按钮行为、API 调用、结果预览。
minimap.js	小地图导航。	模块/节点折叠状态在小地图显示。
python_snippet_ide.js	PythonSnippet 专用 IDE 面板。	变量说明、模板、校验结果。
auth_workspace.js	登录、注册、退出、工作区切换、在线用户心跳。	多人在线和权限头。
bootstrap.js	页面初始化、事件绑定。	新增入口按钮或初始化流程。
styles.css	整体样式、模块卡片、面板、弹窗。	视觉调整。

11.1 前端 API 调用约定

frontend/assets/js/api.js 是 API 封装层。登录后应自动带 Authorization: Bearer token；切换工作区后应带 X-Workspace-Id。若出现“我明明登录了但看不到项目/模型”，优先在浏览器 Network 面板确认请求头是否正确。

API 请求头应包含： Authorization: Bearer <token> # 登录后 X-Workspace-Id: ws_xxx # 当前工作区 未登录兼容模式可能包含： X-User-Id: default X-User-Email: ...

11.2 模块化 workflow UI

能力	说明
框选创建模块	在画布空白区域鼠标左键拖拽框选节点，弹出模块配置。
样式配置	标题、说明、颜色、透明度。
折叠/展开	模块顶部 / 按钮切换；折叠后内部节点隐藏。
折叠模块拖动	移动模块卡片时同步移动内部隐藏节点，展开后位置一致。
模块级连线	折叠后内部连线隐藏，对外连线重映射到模块边缘。
结构树	左侧“结构”面板显示 workflow 名称、模块、节点、If/Else 分支。普通连线只排序，不产生缩进。

11.3 workflow 结构树显示原则

结构树不是执行依赖树。普通链路 A -> B -> C 只代表执行顺序，不代表父子关系。因此同级模块/节点应对齐；只有模块包含关系和 If/Else 分支产生上下级缩进。

正确显示： workflow 名称 模块 A #1 ReadImage #2 ORBFeatureMatch #3 Threshold If / Else 选择 If 分支 #4 真分支节点 Else 分支 #5 假分支节点 不建议显示： ReadImage ORBFeatureMatch Threshold FindContours

11.4 前端状态对象维护要点

状态/对象	说明	常见问题
nodes	当前工作流所有节点，包含 id/type/position/params/collapsed。	导入 JSON 后节点未出现，检查 nodes 是否为空或 type 无效。
edges	当前工作流所有连线。	连线消失或端口错误，检查 sourceHandle/targetHandle。
workflowModules	模块列表，保存 node_ids、标题、颜色、折叠状态。	折叠显示异常，检查模块包含的 node_ids 是否仍存在。
selectedNodeId / selectedModuleId	右侧属性面板显示依据。	属性面板错位通常是选中状态未清理。
nodePreviews	运行后节点预览结果。	预览不刷新，检查 /api/run previews 与 render 调用。
currentProjectId	当前打开的服务端项目 ID。	导入 JSON 后如果仍带旧 project_id，可能误保存到旧项目。
authState / currentWorkspaceId	登录用户、token、当前工作区。	项目/模型看不到时检查该状态和请求头。

12. API 与权限维护手册

所有业务 API 都在 /api 下。排查 401/403/404/500 时，先确认路由路径、HTTP 方法、请求头、工作区权限和 payload。

12.1 认证 API

```
POST /api/auth/register
body: {email, password, display_name?}
return: public user
```

```
POST /api/auth/login
body: {email, password}
return: {token, user, expires_at}
```

```
POST /api/auth/logout
GET /api/auth/session
```

12.2 工作区 API

```
GET /api/users/me
GET /api/workspaces
POST /api/workspaces
GET /api/workspaces/{workspace_id}
POST /api/workspaces/{workspace_id}/members
DELETE /api/workspaces/{workspace_id}/members/{member_user_id}
POST /api/presence/heartbeat
GET /api/workspaces/{workspace_id}/presence
```

12.3 项目、模型、运行和任务 API

```
项目：
GET /api/projects
POST /api/projects
GET /api/projects/{project_id}
DELETE /api/projects/{project_id}
GET /api/projects/{project_id}/versions
GET /api/projects/{project_id}/versions/{version_id}
GET /api/projects/{project_id}/diff
POST /api/projects/{project_id}/rollback/{version_id}
POST /api/projects/{project_id}/release/{version_id}

模型：
POST /api/upload-model
GET /api/models
GET /api/models/assets
PATCH /api/models/{model_id}
DELETE /api/models/{model_id}
POST /api/models/cache/clear

运行：
POST /api/run
GET /api/runs
GET /api/runs/{run_id}
POST /api/jobs/run
GET /api/jobs
GET /api/jobs/{job_id}
```

13. 独立脚本导出机制

script_exporter.py 将 workflow 编译成独立 Python/OpenCV 脚本。它并不直接复用运行时算子，而是为常见算子生成对应代码。因此新增算子后，如果要支持“导出脚本”，必须补导出编译逻辑。

函数	用途
generate_algorithm_script	生成核心算法函数。
generate_standalone_script	生成可命令行运行的完整脚本。
_compile_operator	根据节点类型生成代码片段。
_compile_orb_feature_match_for_export	ORB 节点专用导出逻辑。
_compile_python_snippet_for_export	PythonSnippet 导出逻辑。
CUSTOM_SCRIPT_COMPILERS	自定义插件可提供脚本编译器。

运行时 If/Else 是动态分支执行；导出脚本也应尽量保持语义一致。新增复杂控制节点时，需要特别检查脚本导出是否提前计算了不该执行的分支。

14. 自定义算子插件机制

backend/custom_operators/ 目录支持自定义算子。插件加载失败不会导致平台启动失败，错误会在 /api/operators 中返回，方便前端提示。

```
# backend/custom_operators/mean_brightness.py 示例结构
TYPE_NAME = "MeanBrightness"
CATEGORY = "图像"
GROUP = "自定义算子"
METADATA = {...}

def op_mean_brightness(inputs, params, context):
    ...
    return {"out": image, "metrics": {"mean": value}}

def compile_script(...):
    ... # 可选，支持导出脚本
```

适合插件的场景	适合内置公共图像节点的场景
客户/项目专用算法；试验性算法；不想改主注册表。	所有团队都可能复用；需要稳定维护；要纳入公共节点库和文档。

15. 测试体系与回归命令

当前源码已通过 `pytest`：76 passed, 2 warnings。任何涉及执行器、算子、权限、模型、项目或前端结构的修改，都应该至少运行相关测试，提交前运行全量测试。

测试文件	覆盖范围
test_workflow_engine.py	拓扑排序、执行、错误定位、预览。
test_logic_nodes.py	常量、比较、布尔、If/Else、循环逻辑。
test_batch_workflow.py	批量读取、保存结果、CSV。
test_script_exporter.py	独立脚本导出。
test_yolo_operator.py / test_yolo_roi_workflow.py	YOLO 推理输出、目标筛选、ROI 距离。
test_orb_feature_match.py	ORB 模板匹配和 ROI/Mask 映射。
test_python_snippet_node.py	代码片段执行、安全校验、输出。
test_project_and_model_management.py	项目保存、模型列表/删除/缓存。
test_team_platform_features.py	团队平台数据集、回归、运行历史、任务。
test_multiuser_workspace_isolation.py	登录、工作区、项目/模型隔离。
test_frontend_layout.py	前端资源、HTML/JS 布局、关键控件存在性。
<pre># 全量测试 pytest -q # 只测新增图像节点 pytest -q tests/test_my_new_operator.py # 只测多用户隔离 pytest -q tests/test_multiuser_workspace_isolation.py # 前端 JS 语法可用 node --check 做快速检查 node --check frontend/assets/js/app/canvas.js node --check frontend/assets/js/app/actions.js</pre>	

16. 快速故障定位指南

16.1 后端运行失败定位顺序

症状	优先检查	常见修复
页面打不开 / 404	run.py 是否启动；frontend/index.html 是否存在；StaticFiles 路径。	确认在项目根目录运行，检查 FRONTEND_DIR。
/api/operators 500	metadata.py 语法；registry.py 导入；自定义插件错误。	跑 <code>python -m py_compile</code> ；检查 FAILED_CUSTOM_PLUGINS。
节点库缺失某节点	OPERATOR_CATALOG、OPERATOR_META、registry_BUILTIN_OPERATORS。	确保 type 名完全一致。
运行 workflow 失败但前端只看到错误	响应 failed_node、previews、executed_order。	根据 failed_node.type 定位对应 op 函数。
FastAPI JSON 序列化失败	metrics/data 是否包含 ndarray、Path、不可序列化对象。	用 _json_safe_value 规则修正输出。
批量流程不运行	BatchReadImages/SaveBatchResults 数量和连线。	按批量流程约束重连。

症状	优先检查	常见修复
YOLO 模型加载失败	ultralytics 是否安装；model_path 是否存在；工作区权限。	安装依赖、上传模型、确认 X-Workspace-Id。
403 权限错误	用户是否属于工作区；角色是否包含权限。	检查 workspaces/<id>.json 和请求头。
项目/模型看不到	当前工作区是否切换；Authorization 和 X-Workspace-Id。	打开浏览器 Network 检查请求头。
异步任务一直 pending	JobQueue 线程是否异常；任务函数是否抛错未记录。	查看后端日志和 /api/jobs/{id}。

16.2 前端 UI 问题定位

症状	检查文件	定位方法
点击按钮无反应	actions.js / bootstrap.js	浏览器 Console 是否报函数未定义；按钮 id 是否匹配。
节点拖不出来	operators.js / canvas.js	/api/operators 是否返回该节点；拖拽事件是否绑定。
模块折叠后连线不对	canvas.js	检查折叠模块端点重映射逻辑和 hidden node 判断。
结构树缩进错误	canvas.js	普通连线不应形成层级；只对模块和 If/Else 分支缩进。
Python IDE 不显示变量	python_snippet_ide.js	选中节点 type 是否为 PythonSnippet；端口连接读取是否正确。
工作区在线用户不刷新	auth_workspace.js	presence heartbeat 是否定时发送；GET presence 是否 403。
保存项目失败	actions.js / state_store.js / projects.py	Network 面板看 POST /api/projects 的请求和响应。

16.3 节点执行错误分析模板

当 /api/run 返回 success=false 时，按这个顺序排查：
1. failed_node.node_id 哪个节点失败？
2. failed_node.type 对应哪个算子文件？
3. failed_node.error 输入缺失、参数非法、模型错误、图像为空？
4. executed_order 失败前哪些节点已执行？
5. pending_nodes 哪些节点因失败未执行？
6. previews 失败前结果是否正常？
7. 后端日志 是否有完整 Python traceback？
8. 节点 params 和 inputs 与 metadata 中端口要求是否一致？

16.4 常见异常与原因

错误信息关键词	可能原因	处理
流程中存在环	连线形成循环。	删除反向边或使用显式逻辑节点。
存在无效连线	edge.source/target 节点不存在，多发生于导入旧 JSON。	清理 JSON 或前端重连。
缺少输入端口	required 输入没有连接。	检查 metadata inputs.required 与实际连线。
上游节点不存在输出端口	sourceHandle 写错或上游没返回该键。	检查上游 result 字典和 outputs metadata。
算子返回值必须是字典	op 函数直接返回 image。	改成 return {"out": image}。
data 必须是字典 / metrics 必须是字典	返回类型不符合执行器要求。	metrics/data 用 dict。
登录状态已失效	token 过期或被删除。	重新登录。
当前角色没有执行该操作的权限	角色缺少 edit/run/model/manage。	Owner/Maintainer 调整成员角色。
模型不属于当前工作区	YoloInfer 使用了其他工作区模型路径。	切换工作区或重新上传模型。

17. 生产化建议与技术债清单

当前 v14.4 已支持多用户长期试用的基础能力，但存储、任务队列和代码沙箱仍是轻量实现。若要承担正式生产平台，应继续演进。

领域	当前实现	生产建议
用户/会话	文件系统 users.json/sessions.json, PBKDF2 + Bearer token。	接入企业 SSO/LDAP/OAuth；会话存 Redis；审计登录。
项目/版本	文件系统 JSON。	迁移 PostgreSQL；加乐观锁、事务、变更审计。
模型文件	本地 models/uploads。	对象存储或共享文件系统；模型引用计数；模型发布审批。
任务队列	进程内 ThreadPool。	Redis + Celery/RQ/Dramatiq；支持取消、重试、进度、worker 扩容。
在线状态	进程内 presence。	Redis TTL，支持多进程/多实例。
PythonSnippet	AST 限制 + 子进程。	容器沙箱、资源限额、网络隔离、代码审批。
前端	原生 JS。	逐步 TypeScript 化，组件化画布/属性面板/项目管理页。
日志监控	主要依靠控制台和文件。	结构化日志、Prometheus、Sentry、GPU/磁盘告警。

17.1 维护者日常检查清单

- 每次改算子：跑对应算子测试 + /api/operators 检查元数据 + 简单工作流实践。
- 每次改执行器：跑 test_workflow_engine、test_logic_nodes、test_batch_workflow、test_python_snippet_node。
- 每次改权限：跑 test_multiuser_workspace_isolation，并手动用两个用户切换工作区验证。
- 每次改前端画布：node --check 关键 JS 文件，浏览器测试模块折叠/拖动/连线/结构树。
- 每次改模型管理：用不同工作区分别上传模型，确认跨工作区无法引用。
- 每次发布前：pytest -q；确认 README/docs 是否与实际行为一致；打包前清理临时测试数据。

18. 附录 A：后端函数/类索引

下面的索引用于快速定位源码。它不是替代源码阅读，而是帮助你遇到问题时迅速缩小范围。

文件	关键类/函数	定位用途
backend/algorithms/helpers.py	to_int, to_float, ensure_odd, ensure_gray, ensure_bgr, kernel_from_params	后端基础模块
backend/algorithms/image/color.py	op_color_convert, op_gray, op_bgr_to_rgb, op_invert, op_normalize, op_equalize_hist, op_clahe	图像节点运行逻辑
backend/algorithms/image/common.py	gray_input, bgr_input	图像节点运行逻辑
backend/algorithms/image/contours.py	op_find_contours	图像节点运行逻辑
backend/algorithms/image/edges.py	op_canny, op_sobel, op_laplacian	图像节点运行逻辑
backend/algorithms/image/filters.py	op_mean_blur, op_gaussian_blur, op_median_blur, op_bilateral_filter, op_sharpen	图像节点运行逻辑
backend/algorithms/image/geometry.py	op_resize, op_flip, op_rotate90	图像节点运行逻辑
backend/algorithms/image/io.py	op_read_image, op_batch_read_images, op_save_batch_results, op_display	图像节点运行逻辑
backend/algorithms/image/morphology.py	op_erode, op_dilate, op_morph_open, op_morph_close, op_morph_gradient	图像节点运行逻辑
backend/algorithms/image/template_matching.py	op_orb_feature_match	图像节点运行逻辑
backend/algorithms/image/thresholding.py	op_threshold, op_otsu_threshold, op_adaptive_threshold	图像节点运行逻辑

文件	关键类/函数	定位用途
backend/algorithms/logic_ops.py	op_logic_constant, op_logic_compare, op_logic_boolean, op_logic_not, op_logic_if_else, op_logic_for_loop, op_logic_while_loop	后端基础模块
backend/algorithms/operator_registry.py	OperatorDefinition, OperatorRegistry	后端基础模块
backend/algorithms/plugin_loader.py	CustomOperatorDefinition, PluginLoadError, load_custom_operators	后端基础模块
backend/algorithms/python_snippet_ops.py	_SnippetSafetyVisitor, op_python_snippet	后端基础模块
backend/algorithms/types.py	ImageResolver, ExecutionContext	后端基础模块
backend/algorithms/yolo/model_manager.py	normalize_task, is_ultralytics_model_alias, safe_user_id, relative_to_project, CachedYoloModel, ModelCacheView, YoloModelManager, resolve_model_ref, load_yolo_model	YOLO 模型与推理后处理
backend/algorithms/yolo/ops.py	op_yolo_infer	YOLO 模型与推理后处理
backend/algorithms/yolo/roi_distance.py	op_roi_pair_edge_distance	YOLO 模型与推理后处理
backend/algorithms/yolo/selection.py	op_yolo_select_top_detections	YOLO 模型与推理后处理
backend/api/dependencies.py	get_auth_store, get_current_user, get_user_id, get_active_workspace_id, require_workspace_view, require_workspace_edit, require_workspace_run, require_workspace_model, require_workspace_manage, get_image_store ...	HTTP API 与权限依赖
backend/api/routes/auth.py	register, login, logout, session	HTTP API 与权限依赖
backend/api/routes/datasets.py	list_datasets, create_dataset, get_dataset, update_dataset, add_samples, delete_dataset	HTTP API 与权限依赖
backend/api/routes/export_script.py	export_script	HTTP API 与权限依赖
backend/api/routes/jobs.py	list_jobs, get_job, submit_workflow_job	HTTP API 与权限依赖
backend/api/routes/models.py	list_models, list_model_assets, update_model_metadata, delete_model, clear_model_cache	HTTP API 与权限依赖
backend/api/routes/operators.py	list_operators	HTTP API 与权限依赖
backend/api/routes/projects.py	list_projects, save_project, get_project, delete_project, list_project_versions, get_project_version, diff_project_versions, rollback_project, release_project_version	HTTP API 与权限依赖
backend/api/routes/python_snippet.py	PythonSnippetValidationRequest, validate_python_snippet	HTTP API 与权限依赖
backend/api/routes/regression.py	run_regression_test, submit_regression_job	HTTP API 与权限依赖
backend/api/routes/runs.py	list_runs, get_run	HTTP API 与权限依赖
backend/api/routes/team.py	get_me, list_workspaces, create_workspace, get_workspace, upsert_member, remove_member, heartbeat, list_presence	HTTP API 与权限依赖
backend/api/routes/upload.py	upload_image, upload_model	HTTP API 与权限依赖
backend/api/routes/workflow.py	execute_workflow	HTTP API 与权限依赖
backend/custom_operators/_template.py	op_my_operator, compile_script	后端基础模块
backend/custom_operators/mean_brightness.py	op_mean_brightness, compile_script	后端基础模块
backend/main.py	create_app	后端基础模块

文件	关键类/函数	定位用途
backend/models/workflow.py	NodePosition, WorkflowNode, WorkflowEdge, WorkflowModule, WorkflowRequest	后端基础模块
backend/services/auth_store.py	AuthStore	核心服务/持久化/执行
backend/services/dataset_store.py	DatasetStore	核心服务/持久化/执行
backend/services/image_store.py	StoredImage, ImageStore	核心服务/持久化/执行
backend/services/job_queue.py	JobRecord, JobQueue	核心服务/持久化/执行
backend/services/project_store.py	ProjectSummary, ProjectStore	核心服务/持久化/执行
backend/services/run_history_store.py	RunHistoryStore	核心服务/持久化/执行
backend/services/script_exporter.py	generate_algorithm_script, generate_standalone_script	核心服务/持久化/执行
backend/services/team_store.py	WorkspaceStore	核心服务/持久化/执行
backend/services/workflow_diff.py	diff_workflows	核心服务/持久化/执行
backend/services/workflow_engine.py	WorkflowExecutionError, image_to_data_url, topological_sort, run_batch_workflow, run_workflow	核心服务/持久化/执行

19. 附录 B：关键 JSON 样例

19.1 登录与工作区请求

```
# 注册
POST /api/auth/register
{"email": "dev@example.com", "password": "123456", "display_name": "Dev"}

# 登录
POST /api/auth/login
{"email": "dev@example.com", "password": "123456"}

# 后续请求头
Authorization: Bearer <token>
X-Workspace-Id: ws_xxx
```

19.2 项目保存请求

```
POST /api/projects
{
  "project_id": "optional_existing_id",
  "name": "电池外观检测",
  "workflow": {
    "version": "2",
    "nodes": [...],
    "edges": [...],
    "modules": [...]
  },
  "description": "定位 + 缺陷检测 + 逻辑判定"
}
```

19.3 模型元数据修改

```
PATCH /api/models/{model_id}
{
  "asset_name": "battery_defect_detector",
  "model_version": "1.0.3",
  "status": "validated",
  "task": "detect",
  "class_names": ["scratch", "dent", "stain"],
  "metrics": {"recall": 0.982, "false_positive_rate": 0.006},
  "notes": "用于 A 线 2026-07 样本训练"
}
```

20. 附录 C：快速扩展流程图

```
新增公共图像节点
|
|-- 选择 backend/algorithms/image/<category>.py
|+-- 实现 op_xxx(inputs, params, context)
|+-- 用 gray_input/bgr_input 规范图像
|+-- 返回 {out, out_color_space, metrics, data}
|
|-- 修改 backend/algorithms/metadata.py
|+-- OPERATOR_META[type]
|+-- OPERATOR_CATALOG[category][group].items
|
|-- 修改 backend/algorithms/image/__init__.py
|+-- 导出 op_xxx
|
|-- 修改 backend/algorithms/registry.py
|+-- import op_xxx
|+-- _BUILTIN_OPERATORS[type] = op_xxx
|
|-- 可选：修改 backend/services/script_exporter.py
|+-- 支持导出独立脚本
|
|-- 新增 tests/test_xxx.py
|+-- 算法输出
|+-- 注册表可见
|+-- 参数边界
|
|-- 浏览器验证
+-- /api/operators 可见
+-- 节点库出现
+-- 拖拽运行成功
```

21. 附录 D：当前版本确认

项目	状态
源码包	industrial_vision_layout_routing_v14_multiuser_workspace.zip
应用版本	FastAPI title Industrial Vision Workflow, version 2.2.0；产品打包版本 v14.4。
测试命令	pytest -q
测试结果	76 passed, 2 warnings。警告来自 Python 3.13 multiprocessing fork 弃用提示，不影响当前测试结果。
手册用途	用于学习、维护、扩展公共图像节点和排查平台故障。